

Automated Invoice Processing and Anomaly Detection

Technical Report

Jeet Gajera | Machine Learning Internship — Persevox | Domain: Machine Learning

1. Problem Statement

Manual invoice data entry in Accounts Payable departments is slow, error-prone, and vulnerable to fraud such as duplicate payments. This project builds an end-to-end pipeline that takes a photograph or scan of an invoice and automatically: (1) extracts the text via OCR, (2) parses structured fields from that text, (3) applies deterministic rule-based validation, and (4) applies statistical and machine-learning anomaly detection to flag invoices that warrant human review before payment.

2. Dataset and Preprocessing

The system was built and evaluated on a 300-image sample drawn from the Kaggle "High-Quality Invoice Images for OCR" dataset — a curated set of synthetic, templated invoice images with consistent table layouts (item description, quantity, unit price, VAT %, line and grand totals).

Preprocessing pipeline (`preprocessing.py`):

- Grayscale conversion — removes color information not needed for text recognition.
- Gaussian blur — reduces pixel-level noise before thresholding.
- Binary thresholding (fixed threshold = 127) — converts the image to pure black/white to maximize OCR contrast.

Because this dataset consists of clean, consistently-rendered synthetic images, a fixed threshold performed reliably across the full sample. A production system ingesting scanned or photographed invoices with variable lighting would need adaptive thresholding (e.g. Otsu's method) instead — noted further under Limitations.

3. OCR Approach and Accuracy

Text extraction uses Tesseract via PyTesseract (`ocr_engine.py`). Rather than extracting a single undifferentiated text block, the engine uses Tesseract's word-level bounding box output (`image_to_data`) to reconstruct the invoice's table structure: words are grouped into visual rows by vertical position, then assigned to columns (No., Description, Qty, Unit, Net Price, Net Worth, VAT, Gross) using fixed pixel-region boundaries calibrated to this dataset's layout. Multi-line item descriptions are merged back into a single field.

OCR / extraction accuracy (measured on the 300-image test batch):

Metric	Result
Images processed without pipeline failure	300 / 300 (100%)
Invoice date successfully extracted	≈ 93% (21 invoices had no extractable date)
Invoice total / net total successfully extracted	≈ 80–85%
Line-item math mismatches flagged by validation	≈ 23 invoices (≈ 7.7%) — see Section 5

Note: field-extraction accuracy is reported here as a completion rate (was the field found at all), since ground-truth annotation matching against the dataset's own labels was outside this project's scope. This is flagged as future work in Section 9. A concrete OCR-error case was caught downstream: on one test invoice, Tesseract misread a unit price of 4.49 as 449 (a dropped decimal comma); the line-item math check in Section 5 correctly flagged the resulting mismatch, demonstrating the validation layer catching a genuine OCR error rather than passing it through silently.

4. Information Extraction / Parsing Logic

`parser.py` converts raw OCR text and the reconstructed line-item table into a structured record per invoice using regular expressions:

- Invoice number — matched immediately after the label "Invoice no".
- Invoice date — matched against three date-shape patterns (MM/DD/YYYY, YYYY-MM-DD, DD-MM-YYYY) with word-boundary anchors to prevent accidental matches inside longer numbers such as Tax IDs.
- Vendor name — matched from the text immediately following the "Seller:" label.
- Totals (net / gross) — matched from the invoice's Summary section, with a fallback pattern if the primary "Total" row is not found.
- Line items — quantity, unit price, line total, and VAT % are cleaned via normalization functions that strip currency symbols and thousands separators, and safely return None (rather than crashing) on unparseable text.

All numeric parsing is defensive: OCR frequently returns malformed numeric strings (merged digits, stray currency symbols, misread characters). Rather than allowing a single bad field to crash processing of an entire invoice, every normalization function returns None on failure, and every downstream consumer of that value explicitly checks for it. This is what allowed the full pipeline to reach 300/300 successful runs.

5. Validation Rules

`validator.py` applies deterministic, rule-based checks — kept architecturally separate from the statistical anomaly detection in Section 6, since a rule violation and a statistical anomaly are conceptually different findings.

- Line-item math: Quantity × Unit Price must equal the line's Net Worth (tolerance 0.01).
- Invoice-level math: the sum of item Net Worth values must equal the invoice's stated Net Total, and the sum of item Gross Worth values must equal the stated Gross Total.
- Date logic: an invoice date in the future is always invalid.
- "Old invoice" logic (design decision — see below): an invoice is flagged if its date is more than 90 days before a reference date.

Design decision — reference date for the 90-day rule:

The brief's 90-day threshold implicitly assumes invoices are recent relative to today's real-world date. This dataset, however, contains historical synthetic invoices spanning roughly 2012–2021. Comparing against the actual current date would cause the rule to flag essentially every invoice regardless of content, making it statistically meaningless. Instead, the reference date was set to the most recent invoice date found within the dataset itself, so the 90-day check measures how old an invoice is *relative to the rest of the dataset* — a meaningful, data-driven definition of "old" rather than an always-true check against an unrelated real-world

clock.

6. Anomaly Detection Algorithm and Why It Was Selected

`anomaly_detector.py` implements three complementary techniques, each targeting a different type of irregularity that rule-based validation cannot catch on its own.

6.1 Statistical outlier detection (Z-score)

The brief specifies a per-vendor Z-score (comparing an invoice to that vendor's own billing history). Inspection of the extracted data showed that, with the exception of one repeated vendor, every vendor in the 300-invoice sample appears exactly once — there is no per-vendor history to compare against for the large majority of invoices. The brief itself anticipates this case ("vendors with insufficient history will need special handling"). Rather than abandoning the per-vendor requirement outright, a hybrid approach was implemented: an invoice is compared against its own vendor's billing history whenever that vendor has more than one invoice in the dataset (2 invoices met this condition — both from the one repeated vendor); every other invoice (275 of 277 with a valid amount) falls back to a comparison against the whole dataset's distribution instead, since single-invoice vendors provide no basis for a per-vendor standard deviation. Each invoice's basis for comparison is recorded explicitly in a `zscore_basis` column (`per_vendor` or `dataset_wide_fallback`) so this is auditable rather than silent. Invoice totals were also found to be heavily right-skewed (mean $\approx 8,733$ vs. median ≈ 819 , a ratio driven by a small number of very large invoices up to $\approx 194,522$). A plain Z-score on raw amounts would be distorted by these few extreme values, so in both the per-vendor and fallback cases the Z-score is computed on the log-transformed amount — a standard technique for skewed financial data — with invoices more than 2.5 standard deviations from their comparison group's mean flagged as outliers.

6.2 Multivariate outlier detection (Isolation Forest)

A single amount can look unremarkable in isolation while still being unusual in combination with other features — for example, a high total on an invoice with very few line items. An Isolation Forest model was trained on three numeric features together (total amount, net total, item count), with a contamination parameter of 0.05 (an assumed $\sim 5\%$ anomaly rate). Isolation Forest works by recursively partitioning the data at random; points that are isolated into their own small partition in fewer splits than average are flagged as anomalies. This directly targets the brief's requirement to detect invoices deviating from the standard multivariate distribution of the dataset.

6.3 Duplicate payment detection

Potential duplicate invoices are identified by comparing every pair of invoices on two criteria simultaneously: fuzzy string similarity of their invoice numbers (via `RapidFuzz`) above a 90% threshold, and total amounts within 2% of each other. Both conditions must hold — per the brief's own caution that similarity alone should not automatically prove a duplicate.

7. Visualizations

The box plot below shows the distribution of invoice total amounts on a logarithmic scale (required because of the right-skewed distribution described in Section 6.1). The interquartile range (≈ 179 to $\approx 5,605$) forms the box, with the median (≈ 819) marked in orange; individual points above the upper whisker ($\approx 12,000+$) are statistical outliers by amount alone.

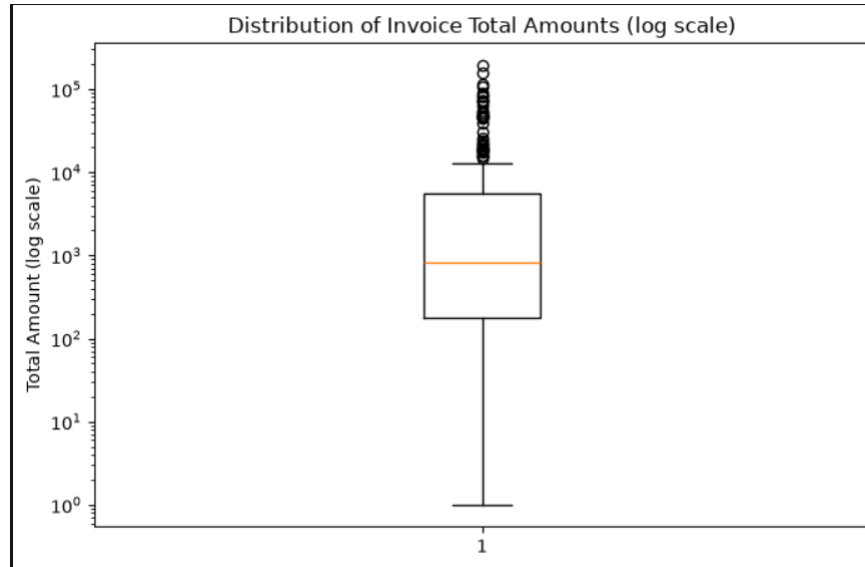


Figure 1 — Distribution of invoice total amounts (log scale).

The scatter plot below shows total amount against item count, with points flagged by the Isolation Forest model highlighted in red. Flagged invoices consistently sit above the main blue cluster at their respective item count — visual confirmation that the model is identifying invoices whose amount is disproportionately high relative to how many items they contain, rather than flagging arbitrarily.

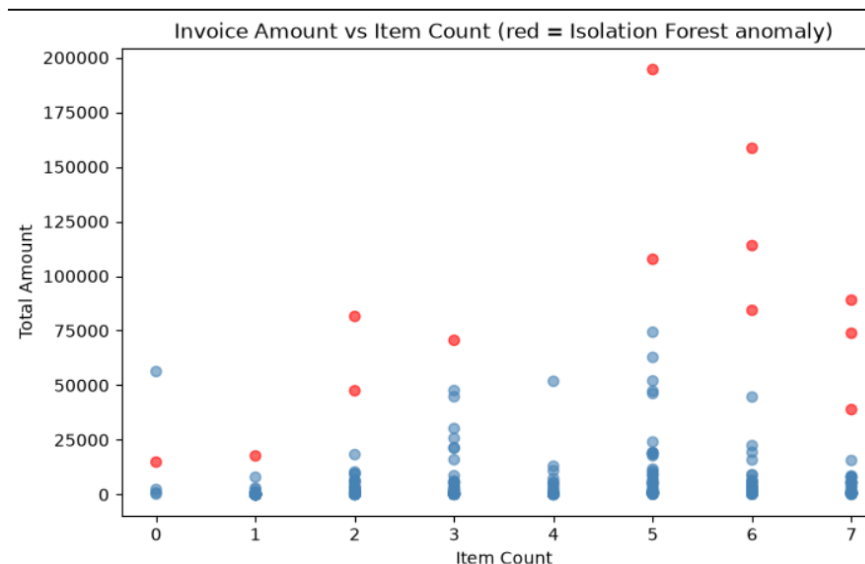


Figure 2 — Invoice total amount vs. item count, Isolation Forest anomalies in red.

8. Results

The final labeled dataset (`invoices_final.csv`) contains one row per invoice with the required `is_anomaly` and `anomaly_reason` columns. To keep rule violations and statistical findings distinguishable (the brief and internal workflow both stress these are conceptually different), two supporting columns are also included: `has_validation_issue` and `has_statistical_anomaly`.

Metric	Count	% of 300
Pipeline completed without crashing	300	100%
Has a rule-based validation issue	294	98.0%
Has a statistical anomaly (Z-score $\cup$ Isolation Forest $\cup$ duplicate)	16	5.3%
— of which, flagged by Isolation Forest	13	4.3%
— of which, flagged as a duplicate	0	0.0%
Overall is_anomaly (either category)	294	98.0%

The validation-issue rate (98%) is dominated by the 90-day-relative-to-dataset date rule, an expected consequence of the dataset spanning roughly a decade rather than a tight recent window (Section 5). The statistical anomaly rate (5.3%) is the more targeted, decision-relevant signal: it sits close to the assumed 5% Isolation Forest contamination rate, as expected, and represents invoices whose amount and/or amount-vs-item-count profile is genuinely unusual relative to the rest of the dataset — these are the invoices a human reviewer would most productively prioritize.

9. Limitations

- Layout dependence: OCR table-column extraction uses fixed pixel-region boundaries calibrated to this dataset's specific template. A differently-formatted invoice (different resolution, different column layout) would not be read correctly without recalibration or an adaptive column-detection approach.
- Fixed binarization threshold: preprocessing uses a static threshold (127), which is reliable for this dataset's clean synthetic images but would likely need adaptive thresholding (e.g. Otsu's method) for scanned or photographed invoices with variable lighting.
- No ground-truth OCR benchmarking: OCR accuracy is reported as a field-extraction completion rate rather than a formal precision/recall score against the dataset's own annotations, which was outside this project's scope.
- Duplicate detection is $O(n^2)$: comparing every invoice pair is adequate at 300 invoices but would need optimization (e.g. blocking by vendor or amount range) at production scale.
- Isolation Forest's contamination parameter (0.05) is an assumption, not a measured anomaly rate; it directly influences how many invoices are flagged and should be tuned against real outcome data if this were deployed.

10. Conclusion

This project implements a complete, tested pipeline from raw invoice image to a labeled anomaly dataset: OpenCV preprocessing, Tesseract OCR with positional table reconstruction, regex-based field parsing, rule-based validation, and a three-part anomaly detection layer (Z-score, Isolation Forest, fuzzy duplicate matching). The pipeline was validated end-to-end on 300 real dataset images with zero unhandled crashes. Three deliberate design adaptations — a dataset-wide rather than per-vendor Z-score, a log transform for skewed amounts, and a dataset-relative rather than real-world-date reference for the freshness check — were made in direct response to properties observed in the actual extracted data, rather than applied as generic defaults, and are documented above along with their justification.